

ML Lab 1: Python Libraries for Machine Learning

NumPy, Pandas, and Matplotlib

Nirjhor Datta

Why Do We Need These Libraries in ML?

Machine learning needs three basic tasks:

- Store and process numerical data
- Load and analyze datasets
- Visualize data and results

Today's Lab

- **NumPy**: numerical computation
- **Pandas**: dataset handling
- **Matplotlib**: visualization

NumPy: Creating Arrays

```
import numpy as np

list1 = [10, 20, 30, 40, 50]
arr1 = np.array(list1)

print(arr1)
# Output: [10 20 30 40 50]

print(arr1.dtype)
# Output: int64
```

Teaching Point

In ML, most data is represented as arrays or matrices.

Creating Arrays with zeros()

```
arr2 = np.zeros(5)

print(arr2)
# Output: [0. 0. 0. 0. 0.]

print(arr2.dtype)
# Output: float64

print(len(arr2))
# Output: 5

print(arr2.size)
# Output: 5
```

Explanation

By default, `np.zeros()` creates floating point values.

Creating a 2D Zero Matrix

```
arr3 = np.zeros(shape=(2, 3), dtype='int')

print(arr3)
# Output:
# [[0 0 0]
#  [0 0 0]]

print(len(arr3))
# Output: 2

print(arr3.size)
# Output: 6
```

Explanation

shape=(2,3) means 2 rows and 3 columns.

2D Arrays: Matrix Representation

```
list1 = [  
    [10, 20, 30],  
    [40, 50, 60]  
]  
  
arr2 = np.array(list1)  
  
print(arr2)  
# Output:  
# [[10 20 30]  
#  [40 50 60]]
```

Meaning

This is a 2D array or matrix.

Rows can represent samples.

Columns can represent features.

Array Addition and Subtraction

```
arr1 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

arr2 = np.array([
    [40, 70, 80],
    [20, 50, 70]
])

print(arr1 + arr2)
# Output:
# [[ 50  90 110]
#   [ 60 100 130]]

print(arr1 - arr2)
# Output:
# [[-30 -50 -50]
#   [ 20  0 -10]]
```

Transpose of a Matrix

```
arr1 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

arr2 = arr1.T

print(arr2)
# Output:
# [[10 40]
#  [20 50]
#  [30 60]]
```

Concept

Transpose converts rows into columns and columns into rows.

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}^T = \begin{bmatrix} 10 & 40 \\ 20 & 50 \\ 30 & 60 \end{bmatrix}$$

Creating Arrays with ones()

```
arr1 = 10 * np.ones(5)  
  
print(arr1)  
# Output: [10. 10. 10. 10. 10.]
```

Explanation

`np.ones(5)` creates five 1s.
Multiplying by 10 changes every value to 10.

Using arange()

```
# range(start, stop exclusive, step)
arr1 = np.arange(10, 21, 2)

print(arr1)
# Output: [10 12 14 16 18 20]
```

- Start = 10
- Stop = 21, but 21 is excluded
- Step = 2

Using arange() with Decimal Step

```
arr1 = np.arange(10, 21, 0.5)

print(arr1)
# Output:
# [10.  10.5 11.   11.5 12.   12.5 13.   13.5
#  14.   14.5 15.   15.5 16.   16.5 17.   17.5
#  18.   18.5 19.   19.5 20.   20.5]
```

Important

Here the values increase by 0.5 each time.

Shape and Size

```
arr2 = np.array([
    [40, 70, 80],
    [20, 50, 70]
])

print(arr2.shape)
# Output: (2, 3)

print(arr2.shape[0])
# Output: 2

print(arr2.shape[1])
# Output: 3
```

Output Meaning

- shape: number of rows and columns
- shape[0]: rows
- shape[1]: columns

Identity Matrix

```
idn = np.eye(5)

print(idn)
# Output:
# [[1.  0.  0.  0.  0.]
#  [0.  1.  0.  0.  0.]
#  [0.  0.  1.  0.  0.]
#  [0.  0.  0.  1.  0.]
#  [0.  0.  0.  0.  1.]]
```

Explanation

An identity matrix has 1 on the diagonal and 0 elsewhere.

Element-wise Multiplication

```
arr1 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

arr2 = np.array([
    [40, 70, 80],
    [20, 50, 70]
])

print(arr1 * arr2)
# Output:
# [[ 400 1400 2400]
#   [ 800 2500 4200]]
```

Important

- * multiplies matching positions.

Matrix Multiplication

```
arr1 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

arr2 = np.array([
    [40, 70],
    [20, 50],
    [80, 70]
])

print(arr1 @ arr2)
# Output:
# [[3200 3800]
#  [7400 9500]]

print(arr1.dot(arr2))
# Output:
# [[3200 3800]
#  [7400 9500]]

print(np.matmul(arr1, arr2))
```

Broadcasting

```
arr2 = np.array([
    [40, 70],
    [20, 50],
    [80, 70]
])

arr3 = np.array([10, 20])

print(arr2 * arr3)
# Output:
# [[ 400 1400]
#   [ 200 1000]
#   [ 800 1400]]
```

Explanation

The array [10, 20] is applied to every row of arr2.

Adding a Scalar to an Array

```
arr2 = np.array([
    [40, 70],
    [20, 50],
    [80, 70]
])

arr3 = arr2 + 5

print(arr3)
# Output:
# [[45 75]
#  [25 55]
#  [85 75]]
```

Explanation

The scalar value 5 is added to every element.

Flatten and Ravel

```
arr2 = np.array([
    [40, 70],
    [20, 50],
    [80, 70]
])

arr3 = arr2.flatten()

print(arr3)
# Output: [40 70 20 50 80 70]

print(arr2.ravel())
# Output: [40 70 20 50 80 70]
```

Difference

- `flatten()` creates a new copy
- `ravel()` usually returns a view

Pandas: Why Do We Need It?

Pandas is used for handling tabular datasets.

- CSV files
- Excel-like tables
- Rows and columns
- Missing values
- Data statistics

In ML

Before training a model, we usually inspect and clean data using Pandas.

Pandas Series

```
import pandas as pd

list1 = [10, 20, 30, 40, 50]
ds = pd.Series(list1)

print(ds)
# Output:
# 0      10
# 1      20
# 2      30
# 3      40
# 4      50
# dtype: int64
```

Concept

A Series is like a single column of data.

Series Index and Values

```
print(ds.index)
# Output: RangeIndex(start=0, stop=5, step=1)

print(ds.values)
# Output: [10 20 30 40 50]
```

Explanation

A Pandas Series has both index labels and values.

Custom Index in Pandas Series

```
idx = ['a', 'b', 'c', 'd', 'e']
list1 = [10, 20, 30, 40, 50]

ds = pd.Series(list1, index=idx)

print(ds)
# Output:
# a      10
# b      20
# c      30
# d      40
# e      50
# dtype: int64
```

Meaning

Instead of default index 0, 1, 2, 3, we can use custom labels.

Reading a Dataset

```
loc = "/content/iris.csv"  
df = pd.read_csv(loc)
```

```
print(df)
```

```
# Output:
```

```
#      sepal.length  sepal.width  petal.length  petal.width  variety  
# 0              5.1          3.5           1.4           0.2     Setosa  
# 1              4.9          3.0           1.4           0.2     Setosa  
# ...  
# 149            5.9          3.0           5.1           1.8  Virginica  
#  
# [150 rows x 5 columns]
```

Iris Dataset

The Iris dataset contains flower measurements and flower class labels.

Inspecting Dataset Shape

```
print(df.shape)  
# Output: (150, 5)
```

Meaning

The dataset has 150 rows and 5 columns.

Dataset Information

```
df.info()
```

```
# Output:
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 150 entries, 0 to 149
# Data columns (total 5 columns):
#  #   Column          Non-Null Count  Dtype
# ---  -
#  0   sepal.length    150 non-null    float64
#  1   sepal.width     150 non-null    float64
#  2   petal.length    150 non-null    float64
#  3   petal.width     150 non-null    float64
#  4   variety         150 non-null    object
# dtypes: float64(4), object(1)
```

Teaching Point

`info()` helps us check column types and missing values.

Statistical Summary

```
df.describe()
```

```
# Output:
```

	sepal.length	sepal.width	petal.length	petal.width
# count	150.000000	150.000000	150.000000	150.000000
# mean	5.843333	3.057333	3.758000	1.199333
# std	0.828066	0.435866	1.765298	0.762238
# min	4.300000	2.000000	1.000000	0.100000
# 25%	5.100000	2.800000	1.600000	0.300000
# 50%	5.800000	3.000000	4.350000	1.300000
# 75%	6.400000	3.300000	5.100000	1.800000
# max	7.900000	4.400000	6.900000	2.500000

Meaning

This gives count, mean, standard deviation, minimum, maximum, and quartiles.

Checking Missing Values

```
df.isnull().sum()
```

```
# Output:
```

```
# sepal.length      0
```

```
# sepal.width       0
```

```
# petal.length      0
```

```
# petal.width       0
```

```
# variety           0
```

```
# dtype: int64
```

Why Important?

Missing values can affect ML models.

Checking Class Counts

```
df.value_counts("variety")  
  
# Output:  
# variety  
# Setosa          50  
# Versicolor     50  
# Virginica       50  
# Name: count, dtype: int64
```

Meaning

The dataset is balanced because each class has 50 samples.

Adding a New Column

```
list1 = 150 * [100]

print(list1)
# Output:
# [100, 100, 100, ..., 100]
# Total 150 values

df["New"] = list1
```

Explanation

This adds a new column named `New` where every row has value 100.

Matplotlib: Why Visualization Matters

Visualization helps us understand data.

- See trends
- Detect outliers
- Understand distributions
- Explain results clearly

In ML

Before training models, visualizing data is very useful.

Simple Line Plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([0, 6])
y = np.array([10, 40])

plt.plot(x, y)
plt.show()

# Output:
# A line plot connecting points (0,10) and (6,40)
```

Meaning

A line plot connects points using straight lines.

Line Plot with Marker and Style

```
points = [3, 5, 90, 23, 45]

plt.plot(points, marker="o",
         linestyle="dotted",
         color="r")

plt.show()

# Output:
# A red dotted line plot with circle markers
```

- `marker="o"` shows points
- `linestyle="dotted"` gives dotted line
- `color="r"` makes line red

Plotting a Mathematical Function

```
x = np.arange(-10, 10.25, 0.25)
y = x ** 2

plt.title("y = x * x")
plt.xlabel("values of x")
plt.ylabel("values of y")
plt.grid()
plt.plot(x, y)
plt.show()

# Output:
# A parabolic curve showing y = x^2
```

Concept

Here we visualize the function:

$$y = x^2$$

Scatter Plot

```
x = np.arange(-10, 11, 1)
y = x ** 2

plt.title("y = x * x")
plt.xlabel("values of x")
plt.ylabel("values of y")
plt.grid()
plt.scatter(x, y, color="orange", marker="*")
plt.show()

# Output:
# Orange star-shaped points following y = x^2
```

Line Plot vs Scatter Plot

Scatter plot shows individual data points without connecting them.

Multiple Plots Using subplot()

```
x = np.arange(-10, 11, 1)

plt.subplot(3, 2, 1)
plt.plot(x, x ** 2)

plt.subplot(3, 2, 2)
plt.plot(x, x ** 3)

plt.subplot(3, 2, 3)
plt.plot(x, 2 * x)

plt.subplot(3, 2, 4)
plt.plot(x, np.sin(x))

plt.subplot(3, 2, 5)
plt.plot(x, x - 2)

plt.subplot(3, 2, 6)
plt.plot(x, 3 * x ** 2 + 5)

plt.show()
```

Histogram

```
x = np.random.normal(150, 20, 200)

plt.hist(x)
plt.show()

# Output:
# A histogram of 200 random values
# generated from a normal distribution
# with mean 150 and standard deviation 20
```

Meaning

A histogram shows how values are distributed.

Class Practice Tasks

- 1 Create a NumPy array from 1 to 20.
- 2 Convert a 2D array into a 1D array.
- 3 Load the Iris dataset using Pandas.
- 4 Print the shape, info, and statistical summary.
- 5 Count how many samples exist for each flower variety.
- 6 Plot sepal length vs petal length using Matplotlib.

- NumPy is used for arrays and matrix operations.
- Pandas is used for dataset loading and analysis.
- Matplotlib is used for plotting and visualization.
- These tools are essential before applying ML algorithms.

Thank You